

GIT vs GITHUB: DIFERENCIAS Y PRIMEROS PASOS

Del control de versiones local a la colaboración en la nube — guía práctica del Laboratorio 01

**GIT**

Sistema de control de versiones local

•¿Qué es?

Corre en tu computadora y funciona **sin conexión a internet**. Registra el historial de cambios de un proyecto para poder volver a cualquier "punto de control" guardado.

•Las 4 áreas de Git

DIRECTORIO DE TRABAJO

→

ÁREA DE PREPARACIÓN

→

REPOSITORIO LOCAL

→

REPOSITORIO REMOTO

Cada integrante avanza en su propia copia offline y luego sincroniza con el repositorio remoto en GitHub.

•Comandos esenciales

1

Iniciar el repositorio en la carpeta del proyecto:
`git init`

2

Preparar archivos (área de preparación):
`git add .`

3

Guardar un punto de control con mensaje:
`git commit -m "Mensaje"`

**GITHUB**

Plataforma en la nube para el equipo

•¿Qué es?

Es un servicio en línea que **aloja el repositorio remoto**. Permite compartir el proyecto, colaborar con el equipo y mantener sincronizadas todas las copias locales.

•Conectar terminal con GitHub



Autenticar la terminal con tu cuenta (GitHub CLI):
`gh auth login`



Enlazar el repo local con el remoto creado en GitHub:
`git remote add origin <url>`

•Comandos clave



Subir tus cambios al remoto:
`git push`



Bajar novedades del equipo:
`git pull`



Copiar un proyecto existente:
`git clone <url>`

⚖ Diferencias clave entre Git y GitHub

Aspecto	GIT	GITHUB
Naturaleza	Herramienta / software de control de versiones	Servicio / plataforma en la nube
Dónde vive	Se instala y corre en tu computadora	Se aloja en internet (servidores remotos)
Conexión	Funciona sin internet	Requiere internet para sincronizar
Alcance	Historial y trabajo individual/local	Colaboración y trabajo en equipo
Función principal	Guardar versiones (commits) del proyecto	Compartir, respaldar y sincronizar repositorios
Analogía	Tu "máquina del tiempo" personal	El "espacio compartido" del equipo

💻 Desde la terminal (gh + git)

- 1

`gh auth login` — conecta la terminal con tu cuenta de GitHub (HTTPS o SSH).
- 2

`git init` dentro de la carpeta del proyecto.
- 3

`git add .` y `git commit -m "Initial commit"` — guarda la primera versión.
- 4

Crea un repositorio vacío en GitHub y copia su URL.
- 5

`git remote add origin <url>` — enlaza el repo local con el remoto.
- 6

`git branch -M main` y `git push -u origin main` — primera subida.
- 7

De ahí en adelante: `git add` → `git commit` → `git push` .

🔗 Desde VS Code (sin terminal de Git)

- 1

Abre la carpeta del proyecto: *File* → *Open Folder*.
- 2

En *Source Control* → *Initialize Repository*.
- 3

Edita archivos, escribe el mensaje de commit y haz clic en *Commit* (✓).
- 4

Clic en *Publish Branch* → inicia sesión en GitHub → elige Public o Private.
- 5

VS Code crea `origin` y sube la rama `main` automáticamente.
- 6

Crea nuevas ramas desde la barra de estado para experimentar sin tocar `main` .
- 7

Activa *Git: Post Commit Command* → *push* para que cada commit se suba solo.

🧠 Conceptos clave para entenderse



Repositorio

Carpeta principal del proyecto. Puede ser local (en tu PC) o remoto (en GitHub).



Commit

Un "punto de guardado" con mensaje: una foto del estado del proyecto en ese momento.



Branch

Una rama para probar ideas sin afectar el código principal (`main`).



Push / Pull

Subir (push) o bajar (pull) cambios entre el repositorio local y el remoto.

📄 Bonus: Markdown rápido para tu README.md

Escribes	Obtienes
<code># Título</code>	Título grande (H1)
<code>## Subtítulo</code>	Subtítulo (H2)
<code>**negrita**</code>	Texto en negrita
<code>*cursiva*</code>	Texto en <i>cursiva</i>
<code>- item</code>	Lista con viñetas
<code>`código`</code>	Texto monoespaciado
<code>[texto] (url)</code>	Enlace